claude-windows / d4b5afb5-2f03-4b58-8952-573f1381d166

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d4b5afb5-2f03-4b58-8952-573f1381d166\subagents\workflows\wf_beb40705-90b\agent-a9e0f39b6a92c9790.jsonl`
- SHA-256: `8fc24cb8e3ab79ff967b3d1dd6565c3810123df1aa8b39799c43b051f00311a1`
- Source modified: `2026-06-21T03:18:30+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_beb40705-90b`
- session_id: `d4b5afb5-2f03-4b58-8952-573f1381d166`

Transcript

1. user (2026-06-21T03:17:18.123Z)

Read these KhelSutra docs to capture the CURRENT honest product scope (so UX/i18n work does not overpromise): C:/Users/avidu/Projects/khelsutra/docs/PER_RALLY_SELECTION.md, C:/Users/avidu/Projects/khelsutra/docs/REQUEST_ACCESS_FORM.md, C:/Users/avidu/Projects/khelsutra/docs/README.md, C:/Users/avidu/Projects/khelsutra/README.md, C:/Users/avidu/Projects/khelsutra/NEXT_STEPS.md, C:/Users/avidu/Projects/khelsutra/site/README.md.
Produce a markdown brief:
- What is HONESTLY shipped vs roadmap (duration-first selection works; shot-count is "experimental"; shot-type/player/score "not available yet"; the per-rally console is job-tethered).
- The request-access form fields/options and what they imply about user segments.
- Anything in NEXT_STEPS relevant to UX/localization.
- Constraints the UX/i18n plan must respect (honest framing, the "default works / BYO-training optional" message, privacy/consent/no-minors posture).
Return the brief as your final message. Faithful, no invention.

2. assistant (2026-06-21T03:17:20.945Z)

I'll read the KhelSutra docs to capture the current honest product scope. Let me start by pulling the repo for freshness, then read all six files.

3. user (2026-06-21T03:17:23.779Z)

fatal: Cannot fast-forward to multiple branches.

4. user (2026-06-21T03:17:23.797Z)

```
# Request-Access form ? segmented lead capture + setup discovery

> **Status:** `IN PROGRESS` · **Owner:** Avi · **Created:** 2026-06-20 · **Last updated:** 2026-06-20
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived`
> **Relation:** first tracked work item in the `khelsutra` repo; mirrors the engine repo's
> `PROJECT_DOC_TEMPLATE` discipline (status header · decisions-locked · §6 tracker · DoD).
> Feeds the platform direction in the engine repo's `PLATFORM_ARCHITECTURE.md §3` (StorageBackend /
> ComputeBackend) and `VIDEO_LOCALITY_MODEL.md` ? see §7.

---

## 0. TL;DR
Replace the landing page's `mailto:` "Request access" CTA with a real form that (a)
```

segments **academies vs individual enthusiasts**, (b) asks the two questions that actually decide how we'd serve a user ? **"do you have a GPU machine?"** (compute placement) and **"where do your videos live?"** (storage backend) ? and (c) keeps a free-text message. Submissions are stored in our own **Cloudflare D1** via a small **Worker** (`POST /api/request-access`). The form doubles as PMF + architecture research.

1. Problem & goal
Today the CTA is `mailto:hello@khelsutra.guru` ? zero structure, nothing collected, no segmentation. We want **structured, queryable** interest signals from two distinct audiences, captured the moment someone raises a hand, **without shipping lead data to a third party** (privacy-first ethos). **Goal:** a clean on-brand form whose responses land in our own Cloudflare account, queryable later.

2. Decisions locked
- **Backend = Cloudflare Worker + D1** (owner decision, 2026-06-20). Data stays in our Cloudflare account; queryable; no third party. (Alternatives considered: hosted form (Tally/Formspree ? third-party data), Worker?email-only (no DB), structured `mailto` (no collection). Rejected for the reasons in parens.)
- **Two audience segments:** `academy` and `individual` ? the form lightly adapts (academy also asks club name + #courts).
- **The two discovery questions are first-class** (not optional polish): GPU/compute + storage ? they map 1:1 onto the BYO-compute / BYO-storage architecture forks (§7).
- **Free-text message retained** ("Anything else?").
- **Privacy:** store only what's submitted + a coarse country (from `request.cf.country`); **no raw IP**; consent checkbox for "email me about the alpha"; this data is **operations/contact only**, never a training source (same guardrail as the engine repo).
- **Deploy model changes:** the site moves from dashboard direct-upload ? **Wrangler** (Worker with static assets + D1 binding). Documented in §5; owner runs the `wrangler` deploy (our account/auth).

3. Form spec
Field	Type	Notes
Segment	radio	`academy` | `individual` (required)
Name	text	required
Email	email	required (so we can reply)
City	text	optional
Club / academy name	text	shown for `academy`
Courts recorded	select	`1` | `2-4` | `5+` (academy; optional)
GPU machine for local processing?	radio	`yes` | `unsure` | `no` | `prefer-cloud`
Where do your recordings live / how would you share them?	checkboxes (multi)	`gdrive` | `onedrive` | `dropbox` | `bucket` (S3/GCS) | `local` (SD/drive) | `unsure`
Message	textarea	free text ("Anything else?")
Consent	checkbox	"Okay to email me about the KhelSutra alpha"
`hp`	hidden honeypot	anti-spam; if filled ? silently drop

4. Design / architecture
- **Frontend** (`site/index.html`): the header + hero "Request access" buttons scroll to

````
the
   `#join` form; `#join` becomes the form. On submit, JS `POST`s JSON to `/api/request-access`.
   **Progressive enhancement:** if the endpoint isn't live yet (pre-Worker), it falls back to a
   prefilled `mailto:hello@` so the form is useful from the first re-upload.
- **Worker** (`/api/request-access`): validates (email present, honeypot empty), inserts into D1,
   returns `200 {ok:true}`; serves the static assets for all other routes.
- **D1 schema** (`access_requests`): `id, created_at, segment, name, email, city, org_name, courts,
   has_gpu, storage (csv), message, consent, country, user_agent`.
- **Anti-spam:** honeypot first; add **Cloudflare Turnstile** in P2 if spam appears.
- **Notify (P2):** optional email ping to `hello@` per submission (MailChannels / Cloudflare Email).

## 5. Deploy (owner-run; our Cloudflare account)
```
wrangler d1 create khelsutra-access            # once; copy the database_id into wrangler.toml
wrangler d1 execute khelsutra-access --file=site/schema.sql   # create the table
wrangler deploy                                # ships the Worker + static assets + D1 binding
```
> Until the Worker ships, the form works via the `mailto` fallback (re-upload `site/` as today).
> Git?Cloudflare auto-deploy still intentionally not connected.

## 6. Deliverables & progress tracker  ? source of truth
Legend: ? Todo · ? In progress · ? Done. One PR per row off `origin/master`.
| ID | Deliverable | Gated? | Status | PR |
|----|-------------|--------|--------|----|
| P0 | This design doc + `NEXT_STEPS.md` | No | ? | #2 |
| P1 | Form UI (segments + GPU + storage + free text + consent) w/ `mailto` fallback | No | ? | #2 |
| P2 | Worker + D1 + `wrangler.toml` + `schema.sql` + **unit tests (10 green)** | owner deploy | ? | this PR (built+tested; owner runs `wrangler d1 create` + `deploy`) |
| P3 | Anti-spam (Turnstile) + per-submission email notify | No | ? | ? |
| P4 | Tiny admin/export view (auth'd) to read responses | owner | ? | ? |

## 7. Why this matters beyond lead capture
The two questions are **the architecture forks** from the engine repo's BYO analysis:
- **GPU?** ? compute placement: desktop/local (their GPU) vs hosted vs zero-infra-BYO
   (`PLATFORM_ARCHITECTURE.md §3.2`, `DESKTOP_APP_PLAN.md` P0 spike).
- **Storage?** ? which `StorageBackend` + whether autosync is worth building
   (`PLATFORM_ARCHITECTURE.md §3.1`, the cloud-storage/autosync thread).
So early responses directly inform what to build next ? not just who to email.

## 8. Open questions
1. Turnstile now or wait for spam? (Default: honeypot now, Turnstile if needed.)
2. Email-notify per submission, or just query D1 weekly? (Default: notify in P2.)
3. Do we localize the form (Kannada/Hindi) for academies now or at beta? (Default: English now.)

## 9. Definition of done
- [ ] Form replaces the `mailto` CTA, segments academy/individual, asks GPU + storage + free text.
- [ ] Submissions persist to Cloudflare D1 via the Worker; honeypot drops bots.
- [ ] Privacy posture honored (no raw IP; consent; contact-only).
- [ ] Deployed live on khelsutra.guru; a test submission appears in D1.
- [ ] Doc tracker + DoD updated; archived when P1?P3 land.

````

5. user (2026-06-21T03:17:23.852Z)

khelsutra docs index

Tracked work (this repo)
- **`LOCAL_APPLIANCE_ARCHITECTURE.md`** ? the single-box, UI-powered rally indexer + reel
  generator (KhelSutra Studio): the `web/` SPA drives the full ingest ? index ? reel loop on one
  box behind an edge gate; the single-box deployment profile of the engine's decoupled
  architecture. Umbrella for the console; per-rally selection is one screen of it. `DRAFT`.
- **`PER_RALLY_SELECTION.md`** ? interactive pick-rallies ? reel builder (RallyPicker). The alpha
  SPA over the engine's rally index; hybrid grid + deterministic query bar on the **existing**
  `/api/query` + `/api/compile` contract. `IN PROGRESS` (decisions locked).
- **`REQUEST_ACCESS_FORM.md`** ? segmented lead-capture form + Worker/D1 backend (shipped).
- **`PORTABILITY.md`** ? the Node+SQLite VM-host runbook (Cloudflare ? droplet parity).

Engine planning docs
Canonical planning docs currently live in the engine repo (`badminton-highlight-indexer/docs`):
`UI_PROPOSAL.md` · `HOSTING_PLAN.md` · `UI_ALPHA_EXECUTION_PLAN.md`. They migrate here when the
control plane lands (per ADR-009 in the engine repo's `docs/archives/decisions/DECISIONS.md`).

6. user (2026-06-21T03:17:24.344Z)

KhelSutra Guru ?

Private product repo ? the heart and brain of khelsutra.guru:
the web app (alpha SPA) now, the hosted control plane at beta.

- **Engine** (pipeline, API, eval): sibling repo `badminton-highlight-indexer` (rename pending).
- **Canonical planning docs** (live in the engine repo's `docs/` for now, migrate here at beta):
  `UI_PROPOSAL.md` · `HOSTING_PLAN.md` · `UI_ALPHA_EXECUTION_PLAN.md`.
- **Status:** pre-alpha scaffold. SPA lands in `web/` per execution-plan milestone M1.

Structure

- `site/` ? public marketing landing page (static; served at khelsutra.guru)
- `web/` ? React SPA (alpha; Vite + React + TS + Tailwind; Cloudflare Pages build root)
- `docs/` ? product docs as they migrate from the engine repo
- `control-plane/` ? (beta) Cloud Run FastAPI control plane

License

7. user (2026-06-21T03:17:24.387Z)

Interactive per-rally selection ? reel builder (RallyPicker)

> **Status:** `IN PROGRESS` ? **MVP / M0 (P1?P5) COMPLETE 2026-06-20** (pick ? query ? select ? compile ? download, on existing engine endpoints; verified e2e against a stub engine, GCP-GPU validation next). Design **APPROVED by owner 2026-06-20**. Downstream milestones M1?M5 (multi-video auth, per-rally preview endpoint, corpus loop, localized NL) remain owner-gated. · **Owner:** Avi · **Created:** 2026-06-20 · **Last updated:** 2026-06-20
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when

DONE)
> **Tracking anchors:** §7 progress tracker is the source of truth; indexed in `docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
> **Relation:** second major tracked work item in the `khelsutra` repo (peer of `REQUEST_ACCESS_FORM.md`); mirrors the engine repo's `PROJECT_DOC_TEMPLATE` discipline. Primarily realizes alpha screen **A5 Highlights** (engine repo `UI_PROPOSAL.md:47`, exec-plan **M4** `UI_ALPHA_EXECUTION_PLAN.md:39`); its correction-loop extension realizes **A4 Rally Review** (`UI_PROPOSAL.md:46`, exec-plan **M3** `:38`). Consumes the engine's `/api/query` + `/api/compile` contract.
> **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine code this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not legal/financial advice.

---

0. TL;DR
Build the **pick-rallies ? reel** surface as a **hybrid**: a selectable rally **list/grid** is the workhorse; a **deterministic query bar** ("rallies over 10s, longest first") pre-selects into the **same** selection set; a sticky footer compiles the selection into one downloadable MP4. The entire MVP runs on **existing engine endpoints** ? `POST /api/query` ? `POST /api/compile` ? `GET /api/highlights` `[verified backend/main.py:331-340, 342-396, 307-328]` ? with **zero new engine intelligence and zero new endpoints**. The single most important caveat: queries and chips are scoped to **length / count / order / shot-count only**; shot-type, player, and score are **roadmap-only** (no detector, no column) and must not be faked. The biggest UX-vs-effort tension: **no per-rally preview is possible today** ? only the *final compiled reel* can stream ? so MVP selects on text metadata, and visual preview/timeline is a deliberate later milestone.

1. Problem & goal
Today the alpha can detect rallies and stitch a reel, but reel-building is either a **silent `stitching.min_shot_count` threshold** `[verified backend/main.py:362-383]` or a static checkbox list whose data plumbing is even broken ? the bundled static UI reads `data.rallies` while the API returns `play_segments`, so it renders nothing `[verified backend/api/static/app.js:344 vs backend/main.py:340]`. The owner's framing is **dynamic per-video selection, not a fixed threshold**: a coach should say "give me the long rallies from this session" and get exactly those, then fine-tune by hand.

The concrete outcome ("good"): A coach uploads a 40-minute session; the engine finds 60 rallies. They type *"top 15 longest"*, eyeball the auto-selected cards, untick two warm-up rallies, name it *"Tuesday ? long points"*, and download a ~6-minute reel ? without learning any threshold knob.

Goal: the smallest *honest* surface that turns engine-detected rallies into a user-curated, downloadable reel, with both **fast** (query) and **precise** (checkbox) selection sharing **one** state.

Non-goals (alpha): no conversational LLM (`UI_PROPOSAL.md:66` defers it); no shot-type/player/score filters (no data exists); no per-rally video preview while it requires unbuilt streaming; no multi-tenant auth (single-box alpha).

2. Decisions locked
#	Decision	Source / date	Implication
D1	Dominant surface = selectable rally **list/grid**	Synthesis 2026-06-20 (Grid scored 15, fit-to-data 4)	Scales to long matches via bulk+filter; shows honest fields only
D2	Fast entry = **deterministic** client-side `parseQuery()` (no LLM)	`[verified UI_PROPOSAL.md:66]` conversational QA deferred	Predictable, unit-testable, offline, zero API cost
D3	**One** shared `Set<segment_id>`, mutated by query + checkbox + (future) timeline	Synthesis 2026-06-20	No second compile path can diverge from what the user sees
D4	Compile via existing `POST /api/compile {video_id, segment_ids, output_filename}`	`[verified backend/main.py:342-396]`	Selection order is harmless ? stitcher `_merge_overlapping` de-dupes `[verified compiler.py:80-105, 311-317]`
D5	Query grammar (alpha) = length, count, order, **shot_count only**	`[verified database.py:122-135, gemini.py:470-485]`	Only fields the payload actually returns; grey out

the rest |
D6	MVP backend = **zero new endpoints**	`[verified]` reuse map (§4)	Ships the full loop on the already-shipped contract
D7	Replace the silent `min_shot_count` cut with **explicit user selection**	Owner framing + `[verified main.py:362-383]`	UI *warns* about the floor; it never hides why a rally vanished
D8	**Default selection = ON** (curate-by-removal)	Owner 2026-06-20	Rallies load all-selected (matches today's "all rallies" reel); user removes warm-ups. Footer/empty UX assumes a non-empty start
D9	**Preview-before-select is NOT a ship-blocker** for the MVP	Owner 2026-06-20	MVP selects on text metadata + previews the *final compiled reel*; the source/clip-stream endpoint (per-rally preview) is the next milestone (P9), not MVP
D10	**MVP is job-tethered** (carry `result.video_id`); no `GET /api/videos`	Owner 2026-06-20	Smallest honest slice; multi-video home + identity scoping is P8 (gated)
D11	`min_shot_count`: **warn, do NOT override** in the MVP	Owner 2026-06-20	The UI surfaces the floor; an explicit-selection override of the compile default is deferred (would trigger the Launch Report convention)

3. Foundation ? what the engine gives us today `[verified this session]`
The whole design rests on the engine's real, already-shipped contract. Read against the engine repo (`badminton-highlight-indexer`):

- **The rally store is `play_segments`** ? columns `id, video_id, start_time, end_time, duration, server, receiver, metadata` `[verified database.py:124-135]`. `id` is the integer the UI selects and passes to compile; `start_time/end_time/duration` are REAL seconds (duration computed at insert).
- **Listing rallies = `POST /api/query {video_id}` ? `{play_segments:[...]}`** ? the *only* rally-list path today; it JSON-parses `metadata` and joins `events` `[verified main.py:331-340, database.py:427-475]`.
- **Building a reel = `POST /api/compile {video_id, segment_ids:[int], output_filename}`** ? loads `videos.filepath`, filters all segments to the requested `segment_ids`, applies the `min_shot_count` floor, extracts `time_pairs=[(start,end)]`, and calls `VideoCompiler.compile_highlights(raw_path, segments, name)` `[verified main.py:342-396, compiler.py:303]`, which **merges overlaps** then per-clip re-encodes ? concat ? one MP4 `[verified compiler.py:311-317]`. Selection order/duplicates are therefore safe.
- **Downloading = `GET /api/highlights/{video_id}/{filename}`** ? streams the compiled MP4. Compile returns a **server-local filepath, NOT a URL** `[verified main.py:394]`, so the SPA constructs this download URL itself. The compiled reel is the **only streamable video that exists today** `[verified main.py:307-328]`.
- **The floor knob is readable** ? `GET /api/config` exposes `stitching.min_shot_count` `[verified main.py:121-123]`, so the UI can warn before a selected rally is silently dropped.
- **Job entry** ? `GET /api/jobs/{job_id}` returns `result.video_id` `[verified main.py:274-293]`, which is how the MVP reaches a video without a (nonexistent) list-videos endpoint.

Honest fields vs fabricated/roadmap (critical for the UI):
- **Honest today** (default `default_segmenter: "gemini"`, per `config.json` `[verified]`): `start_time`, `end_time`, `duration`, and `metadata.shot_count` (Gemini's `shuttle_exchange_count`, fallback `max(3, int(dur/0.8))`) + `metadata.shot_count_confidence` (a **string**, often `"Unknown"`) `[verified gemini.py:470-485]`. On the gemini path `server`/`receiver` are written `None` ("no fabricated data") `[verified gemini.py:480-487]`.
- **Fabricated ? do NOT surface:** the `motion` demo segmenter fills `server`/`receiver` via `random.sample(...)`, a random `shot_count`, and random `events` (serve/smash/foul/winner) `[verified motion.py:191-198]`. `query` returns these columns + `events`, but they are not ground truth and must stay hidden.
- **Roadmap ? no column, no detector:** shot-type per shot, player identity, match/point score, game/set grouping. None exist anywhere in the schema; do not consume them `[verified database.py whole-schema 122-148]`.
- **Correction-loop columns exist but are dead:** `candidate_segments.human_label/notes/confirmed_segment_id` are defined but no endpoint writes them `[verified database.py:166-168]` ? that's the M4 corpus extension (engine PR-4/B6, `UI_ALPHA_EXECUTION_PLAN.md`).

4. Design / architecture

Reuse map (existing ? `[verified]`, no change for MVP): `POST /api/query` (list) ? `POST /api/compile` (stitch) ? `GET /api/highlights/{video_id}/{filename}` (download); `GET /api/config` (floor) ; `GET /api/jobs/{id}` (carry `result.video_id`). **All net-new code is the SPA** in `khelsutra` `web/` (Vite + React + TS + Tailwind, per `web/README.md`), plus a small set of *optional* real-gap engine enablers (§4.3) ? none required for MVP.

4.1 Components (net-new SPA)

- **QueryBar + `parseQuery()`** ? a *pure, deterministic, unit-tested* string ? `{filter, sort}` compiler over `duration` / `ordinal` / `shot_count`. Renders a **parsed-pill** ("filter: duration > 10s · sort: duration desc") so the interpretation is transparent and hand-correctable; greys out shot-type/player/score tokens with an inline "not available yet". No LLM, no network call.
- **RallyList/Grid** ? one card per `play_segment`: ordinal (by `start_time`), `mm:ss` start, duration (`end?start`), and a shot-count + confidence chip **only when `metadata.shot_count` is present** (absent on detector paths that don't write it). Checkbox per card; the thumbnail area is a placeholder (no endpoint yet).
- **`useRallySelection()` hook** ? owns `selectedIds:Set<number>`, `displayOrder`, `apply(queryResult)`, `toggle(id)`, and bulk Select-all / Clear / Invert. Unit-tested (matches the exec-plan's "API client + review-state logic" bar).
- **SelectionFooter (sticky)** ? live "N clips / total M:SS"; a **`min_shot_count` warning** when a selected rally would be dropped at compile (read from `GET /api/config`); reel-name input; one **Build** button.
- **CompileResult** ? `<video src=GET /api/highlights/...>` preview of the *compiled reel* + a download link.
- **TypedApiClient** ? `query`, `compile`, `buildDownloadUrl`; the typed seam to the engine.

4.2 Data flow `[verified anchors inline]`
Gemini segmenter (process time) writes `play_segments {id, start/end_time, duration, metadata.shot_count/confidence/detector}` `[gemini.py:461-498]`
? SPA `POST /api/query {video_id}` ? `{play_segments:[...]}` `[main.py:331-340]` (**fix** the static UI's `data.rallies` read ? `play_segments` `[app.js:344]`)
? normalize to `RallyCard[]` + one shared `selectedIds:Set`; `server`/`receiver`/`events` are present in the payload but motion-only fabrications ? **not surfaced** `[motion.py:191-198]`
? all input modes mutate the **same** `selectedIds`: (a) `parseQuery()` predicate over duration/ordinal/shot_count; (b) checkbox toggle + bulk ops; (c) *(post-MVP)* timeline band
? footer recomputes count + ?duration **client-side**, and warns for any selected rally with `shot_count < stitching.min_shot_count` `[main.py:362-383]`
? **Build** ? `POST /api/compile {video_id, segment_ids: [...selectedIds], output_filename}` ? stitcher de-dupes via `_merge_overlapping` `[compiler.py:311-317]`, cuts `time_pairs` from `videos.filepath`, re-encodes per clip ? concat ? one MP4 `[main.py:386-393, compiler.py:303]`
? compile returns a **server-local filepath, not a URL** `[main.py:394]` ? SPA builds `GET /api/highlights/{video_id}/{output_filename}` to preview in `<video>` + download `[main.py:307-328]`.

4.3 Engine API additions (none for MVP; tagged against real gaps)

Endpoint	Purpose	Status
`POST /api/query`	List rallies (only path) ? fix SPA to read `play_segments`	**exists**
`POST /api/compile`	Stitch selected `segment_ids` ? MP4	**exists**
`GET /api/highlights/{video_id}/{filename}`	Stream/download compiled reel	**exists**
`GET /api/config`	Read `stitching.min_shot_count` for the warning	**exists**
`GET /api/jobs/{job_id}`	Carry `result.video_id` into the picker	**exists**
`POST /api/compile` ? add `download_url`	Return `/api/highlights/...` so SPA stops hand-building it (additive, low-risk; today returns only a filepath `main.py:394`)	**modify**
`GET /api/health`	Engine-offline banner ping the SPA is specified to use (`HOSTING_PLAN.md:42`)	**new**
`GET /api/videos/{video_id}/rallies`	RESTful/cacheable alias returning the same `{play_segments}` shape	**new**

| `GET /api/videos` | List a user's videos for a Match picker (only `get_video(id)`

exists, `database.py:418`) ? needs DB enumerate + identity scoping | **new** |
`GET /api/videos/{video_id}/source` (Range) *or* `/clip?start=&end=`	Stream source/per-span video so a rally previews before selection ? neither exists	**new**
`GET /api/videos/{video_id}/rallies/{segment_id}/thumbnail`	Per-rally poster still (no thumbnail column; WASB frames are transient)	**new**
`PATCH /api/segments/{id}`	Persist accept/reject/nudge ? `candidate_segments.human_label/notes/...` (columns exist, unwritten ? corpus loop)	**new**

? Dead-ends / do-NOT: never surface `server`/`receiver`/`events` (motion-only randoms `[motion.py:191-198]`); never consume shot-type/player/score (no data); never render a confidence **meter** ? `shot_count_confidence` is a Gemini *string* (often `"Unknown"`) and `play_segments` has **no** numeric confidence column.

4.4 Observability (first-class, per `[memory: feedback-launch-quality-bar]`). Every P3 user action emits a **structured JSON log line** (`web/src/lib/log.ts`) so a journey is traceable from text to selection ? the SPA-side "runlog". Each query application generates **one `cid`** shared by every line of that journey; the future `/api/compile` call (P5) reuses the same `cid` to thread the trace into the engine. Events (all on the P3 path):

event	level	when	key fields
`query.parse`	info	every apply attempt	`cid`, `source` (typed/example), `recognized`, `filters`, `sort`, `limit`, `unavailable` ? the exact interpretation
`query.unavailable_request`	info	the query names a shot-type/player/score	`cid`, `raw`, `unavailable[]` ? **demand signal** for roadmap features (what users ask for that we can't yet do)
`query.skip`	info	nothing actionable parsed	`cid`, `raw`, `reason`
`query.apply`	info	a recognized query is applied	`cid`, counts (`total`/`selected`), `selected_ids`, `dropped_unavailable`
`query.empty_result`	**warn**	recognized query matches **zero** rallies	`cid` + full interpretation ? a looks-empty grid is **never silent**
`query.apply_failed`	**error**	the (pure) parser/selector unexpectedly throws	`cid`, `raw`, `error` ? loud, never swallowed
`selection.bulk`	info	Select-all / Clear / Invert	`cid`, `op`, `result_count`
`rally.toggle`	debug	a card is added/removed	`id`, `action`

No silent drops: unsupported concepts (`query.unavailable_request` + `dropped_unavailable`) and empty results (`query.empty_result` warn) are always logged. Verified live in-browser (events fire with correct `cid`/fields, zero console errors).

5. Risk table
Risk	Severity	Mitigation
No source/clip/frame/thumbnail endpoint ? only compiled reels stream `[main.py:307-328]`	High (UX)	MVP selects on text metadata + previews the *final* reel; preview/timeline deferred to M3 behind a new endpoint
`min_shot_count` silently drops selected rallies `[main.py:362-383]`	High (looks broken)	Footer warns *before* Build; D7 makes selection explicit; an override is an owner gate (Launch-Report-worthy `[memory: launch-report-convention]`)
No match-duration field ? a timeline must derive `total = max(end_time)`, mis-rendering pre/post dead air	Med	Timeline deferred to M3; never over-promise it as a true scrubber until source streams
`shot_count` is an *estimate* (`shuttle_exchange_count` w/ `max(3,int(dur/0.8))` fallback) `[gemini.py:470-485]`	Med	Label "approx."; confidence shown as the raw string, not a meter
No auth / user scoping anywhere (CORS `*`, `videos.user_id` never set/filtered)	High for multi-tester	Fine for single-box local alpha; a ship-blocker for any shared host ? gate `GET /api/videos` behind PR-3 scoping
compile returns a filepath not a URL `[main.py:394]`	Low	Client builds the URL for MVP; `download_url` field (M1) removes the brittleness
Surface sprawl (query + grid + future timeline + future correction)	Med	Deliberate IA: query+grid = one screen; correction = separate ? corpus screen (owner call, §8)

6. Honest limits ? what this does NOT do
A green SPA build + passing `parseQuery`/selection unit tests prove the **picker logic

only** ? **not** that the engine's rally boundaries are correct (the measured **rally-END** weakness is a separate quality track, unaffected by this UI). "Conversational" here means a **deterministic four-dimension parser**, not natural-language understanding. Nothing is implemented in `web/` yet ? this is `[design]`. The feature curates *what the engine already found*; it cannot surface a rally the detector missed, nor split/merge a mis-bounded one (that's the M4 correction loop).

7. Deliverables & progress tracker ? **source of truth**
Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**, branched from `origin/master`, no stacking. CI (`npm test`) must stay green; merge to `master` auto-deploys the site (docs/`web/` changes don't touch the live `site/` Worker).

ID	Deliverable	Depends on	Gated?	Status	PR
P0	This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers	?	No	?	[#11](https://github.com/avidullu/khelsutra/pull/11)
P1	SPA scaffold (Vite+React+TS+Tailwind) in `web/` + typed API client; dev proxy to engine `:8000`	P0	No	?	[#14](https://github.com/avidullu/khelsutra/pull/14)
P2	RallyList/Grid (honest fields only) + `useRallySelection()` hook (unit-tested)	P1	No	?	[#21](https://github.com/avidullu/khelsutra/pull/21)
P3	`parseQuery()` + parsed-pill + example chips (unit-tested; greys out unavailable tokens)	P2	No	?	(this PR)
P4	SelectionFooter + `min_shot_count` warning (from `GET /api/config`)	P2	No	?	(this PR)
P5	Build ? `POST /api/compile` ? client-built download URL + `<video>` preview of the reel	P2	No	?	(this PR)
P6	*(engine repo)* `GET /api/health` + offline banner; `/api/compile` returns `download_url`	?	No	?	?
P7	*(engine repo)* `GET /api/videos/{id}/rallies` RESTful alias	?	No	?	?
P8	*(engine repo)* `GET /api/videos` + PR-3 identity scoping ? A1 Matches screen	P7	? owner	?	?
P9	*(engine repo)* source/clip stream + timeline + per-rally preview/thumbnail	P8	? owner	?	?
P10	*(engine repo)* `PATCH /api/segments/{id}` corpus loop (PR-4/B6) ? A4 Rally Review	P9	? owner	?	?

Milestone rollup: **M0/MVP = P1?P5 ? COMPLETE (2026-06-20)** (pick?reel on existing API, zero new engine endpoints) · **M1 = P6?P7** (RESTful + reliability polish) · **M2 = P8** (multi-video home) · **M3 = P9** (per-rally preview/timeline) · **M4 = P10** (corpus correction loop) · **M5** = swap `parseQuery()` for localized NL behind the same seam (gated on shot/player data landing).

8. Open questions ? owner / external
Blocking gates ? ? RESOLVED 2026-06-20 (owner) ? see D8?D11 in §2:
1. ~~Default selection ON vs OFF?~~ ? **ON / curate-by-removal** (D8).
2. ~~Is preview-before-select a ship-blocker?~~ ? **No**; text-metadata MVP, preview is P9 (D9).
3. ~~Job-tethered vs multi-video MVP?~~ ? **Job-tethered** (D10).
4. ~~`min_shot_count` override?~~ ? **Warn, don't override** in MVP (D11).

Still open (nice-to-knows, non-blocking):
5. May user-facing copy call `parseQuery()` "conversational", given conversational QA is explicitly deferred (`UI_PROPOSAL.md:66`)? Risk of over-promising NL. *Owner positioning/honesty call.* *(Lean: avoid the word; show the parsed-pill instead.)*
6. One screen for query+grid (recommended) vs a separate ? corpus/correction screen for A4? *Owner positioning call.*

9. Definition of done
- [ ] From a finished job, a user can list rallies, query/select, and download a reel ? on **existing** endpoints.
- [ ] Only honest fields (ordinal / duration / shot_count+confidence-when-present) are shown; fabricated `server`/`receiver`/`events` are hidden.
- [x] `parseQuery()` covers length/count/order/shot_count, greys out unavailable tokens, shows a visible parsed-pill, and is unit-tested. *(P3)*
- [x] The P3 path is **observable**: structured, correlation-id'd logs for parse / apply

/ skip / empty / unavailable-request / manual-edit; unsupported concepts and zero-result queries are logged loudly, never dropped silently (§4.4). *(P3)*
157 - [x] Selection hook is unit-tested; the `min_shot_count` floor is surfaced, not hidden. *(P2 hook · P4 floor warning ? `lib/floor.ts` mirrors `main.py:362-383`, exempts no-shot_count rallies, blocks Build when all-dropped)*
158 - [x] From a finished job, a user can list rallies, query/select, and download a reel ? on **existing** endpoints. *(P5 ? `?video=<id>` loads via `POST /api/query`; Build ? `POST /api/compile` ? `<video>` preview + download over `GET /api/highlights/...`; verified e2e against a stub engine)*
159 - [x] `npm run build` (and `npm test`) clean; the static UI's `data.rallies` bug is gone ? the SPA **supersedes** the static `app.js` for this flow (reads `play_segments`).
160 - [x] §7 tracker + this DoD updated. *(Doc stays live for the owner-gated M1?M5 engine milestones; not archived ? only the MVP/M0 slice is complete.)*
161
162 ## 10. References
163 **Engine code anchors `[verified this session]`:** `backend/main.py:121-123` (config), `:274-293` (jobs), `:307-328` (highlights download), `:331-340` (query), `:342-396` (compile + `min_shot_count` floor + filepath-not-URL); `backend/infrastructure/database.py:124-135` (play_segments), `:166-168` (dead correction columns), `:418` (no list-videos), `:427-475` (query read path + events join); `backend/pipeline/segmenters/gemini.py:461-498, 470-485` (honest shot_count, server/receiver=None); `backend/pipeline/segmenters/motion.py:191-198` (fabricated fields); `backend/pipeline/stitcher/compiler.py:80-105, 303, 311-317` (merge/de-dupe + entrypoint); `backend/api/static/app.js:344` (`data.rallies` bug); `config.json` (`default_segmenter: "gemini"`).
164 **Engine docs:** `UI_PROPOSAL.md:46-47, 66`; `UI_ALPHA_EXECUTION_PLAN.md:38-39`; `HOSTING_PLAN.md:42`; `OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55` (conversational seam built now, model punted). **This repo:** `web/README.md`, `REQUEST_ACCESS_FORM.md`.
165
166 ---
167 ### Changelog
168 - 2026-06-20 ? Initial `DRAFT`. Consolidated three candidate designs (Timeline / Grid / RallyPicker) into the hybrid recommendation via a multi-agent design pass; every engine-fact anchor re-verified against source before writing.
169 - 2026-06-20 ? Owner **approved** the recommendations; locked D8?D11 (default ON · preview not a ship-blocker · job-tethered MVP · warn-don't-override the floor); status `DRAFT ? IN PROGRESS`; P0 done.
170 - 2026-06-20 ? **UI: lead with rally LENGTH, park shot-count as experimental** (owner). Rally cards emphasize duration (`m:ss long`); the shot-count chip is flagged amber "experimental" with a tooltip ("under development, may be inaccurate ? filter by length instead"); example chips are duration-led (dropped "at least 8 shots"); a note states "rally length is fully supported, shot-count is experimental". `parseQuery` still accepts shot-count queries (not broken) but they're de-featured. Public badge `P5`?`alpha`. A test pins that every example chip parses to a recognized, fully-available (non-shot-count) query. Suite 75 green.
171 - 2026-06-20 ? **P5 shipped ? MVP / M0 COMPLETE.** `App` now loads a real session from `?video=<id>` (`POST /api/query`, default ALL-selected D8) with a mode banner (demo / loading / loaded N / error); **Build** ? `compileReel` (`POST /api/compile`) ? `CompileResult` previews the stitched reel in `<video>` over `GET /api/highlights/{video_id}/{filename}` (client-built URL ? engine returns a filepath, not a URL) + a download link; `reelFilename()` slugifies to an engine-safe bare `.mp4`. Loading/building/done/error states, all traced (`rallies.load_*`, `build.start/done/failed`). Verified e2e against a stub engine: load 5 rallies ? real `/api/config` floor warning ? Build ? `<video>` reached `readyState=4` (decoded a real mp4 through the proxy) + download link correct; all-below-floor selection keeps Build disabled (P4 guard prevents the 400). `reelFilename` unit-tested; suite **73 green**, typecheck + build clean. **Next: GCP-GPU end-to-end validation (real video ? rallies ? reels).**
172 - 2026-06-20 ? **P4 shipped**: `SelectionFooter` (live summary + reel-name input + Build) with an **honest `min_shot_count` warning** ? `lib/floor.ts` mirrors the engine floor (`main.py:362-383`) EXACTLY: a rally is dropped only if it has a numeric `shot_count` below the floor; rallies *without* `shot_count` are exempt; **all-below ? Build disabled** (the engine would 400). Config read from `GET /api/config` on mount (graceful offline). 8 floor unit tests; amber/red warning states verified live in-browser; suite **70 green**, typecheck + build clean.
173 - 2026-06-20 ? **P3 shipped**: deterministic `parseQuery()` (`web/src/lib/query.ts`) over length/count/order/shot_count + `runQuery`/`sortRallies`/label helpers and the `QueryBar` (live parsed-pill, example chips, greyed "not available yet" for shot-type/player/score ? never faked). Writes the shared selection set via a new `useRallySelection().apply`; `RallyGrid` ordinals made stable so a re-sort doesn't renumber cards. **First-class observability (§4.4):**

structured, correlation-id'd `query.*` / `selection.*` / `rally.toggle` runlog (loud
`empty_result`/`apply_failed`; `unavailable_request` demand signal). **Adversarial multi-agent
review** (grammar/honesty/robustness/reuse) ran pre-merge ? 13 confirmed findings folded in,
incl. 4 majors: an honesty leak (`at least 8 smashes` ? fake `length ? 8s`), a selection-wiping
`top 0`, a unit-blind `first 30 seconds`, and self-contradictory pill labels (`first 10` shown
as "top 10"). Live browser CUJ replayed (filter / top-N / honest-greying / untick / no-op /
wipe-closed). Suite **62 green**, typecheck + build clean.
174

8. user (2026-06-21T03:17:24.793Z)

1 # Next steps (live) ? KhelSutra brand / site
2
3 Prioritized "what to do next" for the **`khelsutra` product repo**. Refresh at each
checkpoint.
4
5 **Where we are (2026-06-20):** `khelsutra.guru` is **LIVE and fully shipped** ? landing
page + `/capture`
6 checklist + OG image + a **segmented Request-Access form** (academy/individual + GPU +
storage + free
7 text). On **Cloudflare** (Worker `hidden-king-khelsutra` + D1 `khelsutra-access`): the
form persists to
8 D1 **and emails a summary to the owner on every registration** (Email Routing send
binding ? verified
9 live). **Git auto-deploy is connected** (Workers Builds, root `site/`): merge to
`master` ? deploys;
10 **CI runs `npm test` (20 green) on every PR**. The same form also runs on a dependency-
free **Node +
11 SQLite** host (`site/server.js`), **live + browsable on the DO droplet ?
http://168.144.126.176:8787**
12 (systemd `khelsutra-site`). PRs #1?#8 merged. Docs: `docs/REQUEST_ACCESS_FORM.md`,
`docs/PORTABILITY.md`.
13
14 ## Next product threads (owner ideas, 2026-06-20)
15 - **Marketing video on the site** ? embed a short walkthrough of the record ? index ?
reel process,
16 themed for **serious enthusiasts**, on the landing page. ? **A real-UI CUJ screencast
now exists**
17 (Playwright recording of the per-rally SPA driven against real GPU-run data ?
`promo_cuj.mp4`/`.gif`
18 in `gs://khelsutra-rally-corpus/promo/`). Decide: use it as the marketing clip (?
shows real court
19 footage + a dev "P5" badge ? clean up before public embed), and/or produce a polished
Grok/Gemini cut.
20 - **Interactive per-rally selection** (big product feature) ? ? **MVP / M0 COMPLETE**
(the full
21 pick ? query ? select ? compile ? download loop on existing engine endpoints). Lives
in the alpha SPA
22 (`web/`). **? Tracked design + P-tracker (source of truth):
`docs/PER_RALLY_SELECTION.md` (IN PROGRESS;
23 owner-approved, D8?D11 locked).** **Done: P0 design (#11), P1 scaffold (#14), P2
grid+hook (#21), P3
24 `parseQuery()` query bar (#28), P4 SelectionFooter + `min_shot_count` warning (#30),
P5 real load +
25 Build?compile?preview?download (#31).** Verified e2e vs a stub engine + a CUJ replay
against the real
26 L4-run rallies + reel. **NEXT (owner-gated, engine repo): M1?M5** ? `GET /api/health`
+ `download_url`
27 (P6), RESTful `?/rallies` alias (P7), multi-video home + auth (P8), per-rally
preview/timeline (P9),
28 corpus correction loop (P10), localized NL (M5). **Optional:** a fresh live
`/api/process` GPU
29 inference once the engine session's M2 training frees the single GPU.
30
31 ## Queued (site polish)

```
- **Always-Use-HTTPS + HSTS** (Cloudflare SSL/TLS) ? closes the plain-HTTP flag.
- **Cloudflare Web Analytics** ? paste the token into `site/public/index.html` (or enable auto-setup).
- **`/capture` footer link** ? currently `/capture.html` (307?`/capture`); change to `/capture`.
- **Droplet ? production** ? TLS + a subdomain (`vm.khelsutra.guru` via Caddy) if it should be a real
  standby; add **VM-path SMTP** so the Node host also emails (currently `noopNotify`).
- **DMARC** TXT for deliverability; delete any orphan `khelsutra-site` Worker; clean CF D1 test rows
  (`deploy-test@`, `email-test@`). Revoke the droplet SSH key (`~/.ssh/khelsutra_droplet`) when done.

## Bigger arcs (design-first)
- **Single-box appliance (KhelSutra Studio)** ? **? Tracked design: `docs/LOCAL_APPLIANCE_ARCHITECTURE.md` (DRAFT, owner-gated).**
  The `web/` SPA drives the full **ingest (Drive / bucket URI / local upload) ? index ? pick rallies ? reel ? download**
  loop on ONE box, behind an **edge auth gate** (engine stays `127.0.0.1`, CORS `*` is moot). It is the single-box
  profile of the engine's decoupled `StorageBackend` + `backend.worker` architecture. MVP runs on the CPU droplet via
  **Gemini/stub**; **real local-GPU detection needs a GPU host** (DO droplet / GCE VM / Cloud Run+GPU ? owner priority
  ASAP, runs parallel to the UI MVP). Per-rally selection = one screen. Owner decisions pending in §10 (edge auth,
  retention, cost ceiling, GPU host, P5 bridge).
- **Alpha SPA** in `web/` (Vite + React + TS + Tailwind) ? hosts the operator console + the per-rally selection UI above.
- **BYO-storage + autosync** ? connect Drive/OneDrive/GCS/S3, watch the folder, auto-prepare reels
  (engine `PLATFORM_ARCHITECTURE.md §3`). The form's GPU/storage answers are the demand signal.
- **Control plane** (`control-plane/`) at beta.

## Org & repos
- ? **DONE (2026-06-20): proprietary repos consolidated into the `Khelsutra` org (private).** Transferred
  `badminton-highlight-indexer` (engine), `sports-data-collector`, `sports-obsreport` ? `Khelsutra/*` (stay private; no
  open PRs/forks; GitHub redirects git ops so active sessions kept working; local remotes repointed + verified).
  `rally-annotator` stays in the org **PUBLIC** (OSS MIT plugin the site links to ? do not make private). Owner is org
  admin (full access) and can **invite a contributor** (org member / per-repo outside collaborator) anytime.
- ? **`khelsutra` (this site repo) ? owner step pending.** A transfer breaks the **Cloudflare Workers-Builds** git
  connection (auto-deploy) until the Cloudflare GitHub App is installed on the `Khelsutra` org and the Worker
  reconnected (dashboard). When ready: install the CF app on the org ? `gh api repos/avidullu/khelsutra/transfer -f
  new_owner=Khelsutra` ? reconnect the Worker to `Khelsutra/khelsutra` ? verify a deploy.
- ? **Engine CI under the org** ? verify the engine's GitHub Actions run on the next PR (free orgs default Actions-on;
  enable in org settings if a run doesn't start).

## Convention
Substantial work gets a tracked doc in `docs/`; one PR per item off `origin/master`; merge to `master`
auto-deploys (CI must be green).
```

9. user (2026-06-21T03:17:25.070Z)

````
# site/ ? public marketing site + Request-Access API (Cloudflare Worker)

The public site at **[khelsutra.guru](https://khelsutra.guru)** ? a static landing page +
`/capture` checklist ? plus a small Worker that backs the Request-Access form with **Cloudflare D1**.

## Layout
- `public/` ? static assets served as-is (`index.html`, `capture.html`, `og.png`).
- `src/index.js` ? the Worker: serves `public/` and handles `POST /api/request-access` ? D1.
- `src/index.test.js` ? unit tests for the form logic.
- `wrangler.toml` ? Worker + static-assets + D1 binding config.
- `schema.sql` ? the `access_requests` D1 table.
- `make_og.py` ? regenerates `public/og.png` (Pillow + Windows fonts): `python make_og.py`.

## Develop / test
```
npm test       # node --test src/  (no install needed)
npm run dev    # npx wrangler dev ? local Worker + D1
```

## Deploy (owner ? our Cloudflare account)
One-time:
```
npx wrangler d1 create khelsutra-access                          # paste database_id into wrangler.toml
npx wrangler d1 execute khelsutra-access --remote --file=schema.sql
```
Then either:
- **Manual:** `npx wrangler deploy`, or
- **Git (recommended):** connect this repo in Cloudflare ? Workers Builds (root dir `site/`), bind the
  D1 database ? every merge to `master` deploys; PRs get preview URLs.

## Notes
- The form posts to `/api/request-access`; if the Worker is unreachable it falls back to a prefilled
  `mailto:hello@`, so it degrades gracefully.
- `hello@khelsutra.guru` relies on Cloudflare Email Routing.
- The Web Analytics beacon in `public/index.html` has a placeholder token (`REPLACE_WITH_CF_TOKEN`)
  ? paste the real token or enable Cloudflare automatic setup.

````

10. assistant (2026-06-21T03:18:30.305Z)

The git pull failed (`Cannot fast-forward to multiple branches`), so I'm reading the local checkout. I'll note this in my brief. I have everything I need from the six files. Here is the brief.

KhelSutra ? Honest Product Scope Brief (for UX / i18n planning)

> Source-of-truth read 2026-06-21 against the local `khelsutra` checkout. **Note:** `git pull --ff-only` failed (`Cannot fast-forward to multiple branches`) ? these reflect the **local** working copy, which may lag remote. No remote-truth re-verification was possible. Every claim below is traceable to one of the six docs; nothing is invented.

1. What is HONESTLY shipped vs roadmap

Shipped (alpha, MVP / M0 ? COMPLETE 2026-06-20)
The **per-rally selection SPA** (RallyPicker) in `web/` runs the full **pick ? query ? select ? compile ? download** loop on **existing engine endpoints only** ? zero new engine intelligence, zero new endpoints (`PER_RALLY_SELECTION.md` §0, §7; PRs P0?P5 / #11,#14,#21,#28,#30,#31). Verified e2e against a stub engine + a CUJ replay against real L4-GPU-run rallies and a real reel. GCP-GPU end-to-end validation is the stated next check.

What actually works for a user today:
- **Duration-first rally selection.** Rally cards lead with **length** (`m:ss long`); a deterministic, client-side `parseQuery()` ("top 15 longest", "rallies over 10s") pre-selects into a shared selection set. **"Rally length is fully supported"** is the honest headline (changelog 2026-06-20).
- **Count / order / ordinal** selection (e.g. "top N", "first N") ? fully supported query dimensions.
- **Curate-by-removal:** rallies load **all-selected by default** (D8); user unticks warm-ups, names the reel, and Builds.
- **Compile + download:** Build ? `POST /api/compile` ? `<video>` preview of the **compiled reel** + download via `GET /api/highlights/...`.
- **Honest `min_shot_count` floor warning:** the footer *warns* before a selected rally would be dropped (it does **not** override ? D11); `lib/floor.ts` mirrors the engine floor exactly and disables Build if all selected rallies fall below it.

Experimental (shipped but explicitly de-featured)
- **Shot count.** `parseQuery` still accepts shot-count queries (not broken), but the chip is flagged **amber "experimental"** with a tooltip: *"under development, may be inaccurate ? filter by length instead."* The shot-count value is `metadata.shot_count` = Gemini's `shuttle_exchange_count` with a `max(3, int(dur/0.8))` **fallback estimate**; confidence is a raw **string** (often `"Unknown"`), never a numeric meter. Example chips deliberately avoid shot-count. Labeled "approx." (`PER_RALLY_SELECTION.md` §49, §91, §114; changelog 2026-06-20).

Roadmap ? NOT available yet (no detector, no DB column ? must never be faked)
- **Shot-type per shot, player/athlete identity, match/point score, game/set grouping** ? none exist anywhere in the schema. The query bar **greys these tokens out** with an inline **"not available yet"** and logs them as a demand signal (`query.unavailable_request`); they are never silently dropped (§0, §51, §4.4).
- **Fabricated fields that must stay hidden:** `server` / `receiver` / `events` are written by the **demo `motion` segmenter** via `random.sample(...)` ? not ground truth. On the real (Gemini) path `server`/`receiver` are `None` ("no fabricated data"). The UI must never surface them (§50, §91).

Key shipped constraints (architecture, not just copy)
- **The per-rally console is job-tethered (D10).** The MVP reaches a video only by carrying `result.video_id` from a finished job (`?video=<id>`); there is **no** `GET /api/videos` list, **no** multi-video home, **no** auth/identity scoping yet. Those are owner-gated milestones P8+.
- **No per-rally preview today (D9).** Only the **final compiled reel** can stream; there is no source/clip/thumbnail endpoint. The MVP selects on **text metadata only**; visual preview + timeline are a deliberate later milestone (P9, owner-gated).
- **"Conversational" is a deterministic 4-dimension parser**, not NL understanding. Open question #5 leans toward **avoiding the word "conversational"** in user copy (show the parsed-pill instead) to not over-promise.
- Downstream milestones **M1?M5** (multi-video auth, per-rally preview endpoint, corpus correction loop, localized NL) all remain **owner-gated**.

Marketing site ? separately, fully shipped
`khelsutra.guru` is **LIVE**: landing page + `/capture` checklist + OG image + the segmented Request-Access form, on Cloudflare (Worker + D1), with git auto-deploy and CI (`npm test`, 20 green). Also runs on a dependency-free Node+SQLite host on a DO droplet (`NEXT_STEPS.md`).

2. Request-Access form ? fields/options and the user segments they imply

Form spec (`REQUEST_ACCESS_FORM.md` §3):

Field	Type	Options

Segment	radio (required)	`academy` | `individual`
Name	text	required
Email	email	required
City	text	optional
Club / academy name	text	shown only for `academy`
Courts recorded	select	`1` | `2-4` | `5+` (academy; optional)
GPU machine for local processing?	radio	`yes` | `unsure` | `no` | `prefer-cloud`
Where do recordings live / how would you share them?	checkboxes (multi)	`gdrive` | `onedrive` | `dropbox` | `bucket` (S3/GCS) | `local` (SD/drive) | `unsure`
Message	textarea	free text
Consent	checkbox	"Okay to email me about the KhelSutra alpha"
`hp`	hidden honeypot	anti-spam

What the fields imply about user segments / UX:
- **Two explicit audiences:** `academy` vs `individual`. The form **lightly adapts** ? academies also get club name + #courts (1 / 2-4 / 5+), implying a multi-court, higher-volume, possibly less-technical operator persona vs the solo enthusiast. UX/i18n should plan for **both**, with academies the likelier localization target (see §3).
- **GPU question = compute-placement fork.** `yes` / `unsure` / `no` / `prefer-cloud` maps 1:1 onto **BYO-compute** (their GPU) vs hosted vs zero-infra. A large `unsure`/`no` share signals a **non-technical** user base who can't answer hardware questions ? UX must not assume technical literacy.
- **Storage question = StorageBackend fork.** Where videos live (Drive/OneDrive/Dropbox/bucket/local/unsure) decides which storage backend + whether autosync is worth building. These answers are an explicit **demand signal** for what to build next.
- Open question #3 in the form doc: **"Do we localize the form (Kannada/Hindi) for academies now or at beta?"** ? current default is **English now**, localize at beta. This is directly in scope for the i18n plan to revisit.

3. NEXT_STEPS items relevant to UX / localization

- **Form localization is an open, deferred decision** (`REQUEST_ACCESS_FORM.md` open Q#3): Kannada/Hindi for academies ? default English now, revisit at beta. The i18n plan can take a position here.
- **Marketing video / walkthrough** themed for **"serious enthusiasts"**: a real-UI CUJ screencast exists (`promo_cuj.mp4`) but **shows real court footage + a dev "P5" badge ? must be cleaned up before any public embed.** (The public badge was already changed `P5`?`alpha` in the SPA per `PER_RALLY_SELECTION.md` changelog, but the recorded video still shows the old badge.)
- **Localized NL query is milestone M5** ? swapping the deterministic `parseQuery()` for **localized natural language** behind the same typed seam ? but it's **gated on shot/player data landing** and is the *last* milestone (`PER_RALLY_SELECTION.md` §7 / `NEXT_STEPS.md`). i18n of the *current* UI (static strings, the parsed-pill, honest labels) is independent of and ahead of this.
- **Single-box appliance "KhelSutra Studio"** (`LOCAL_APPLIANCE_ARCHITECTURE.md`, DRAFT, owner-gated) is the umbrella the console lives in: ingest (Drive / bucket URI / local upload) ? index ? pick rallies ? reel ? download on one box. Per-rally selection is **one screen** of it. Relevant as the future home of the UX.
- **Persona anchor for UX tone:** site marketing is explicitly themed for **"serious enthusiasts"** ? but the academy segment + the non-technical GPU/storage answers point to a broader, less-technical audience the UX/i18n work targets.

4. Constraints the UX / i18n plan MUST respect

1. **Honest framing is non-negotiable.** Show only honest fields (ordinal / duration / shot-count-when-present + its string confidence). **Never** surface fabricated `server`/`receiver`/`events` (motion-demo randoms). **Never** consume or imply shot-type / player / score ? grey them out as **"not available yet"**, never fake them. This survives translation: every localized string must preserve the honesty, not soften "experimental"/"not available yet" into something aspirational.
2. **Duration is the hero; shot-count is "experimental"; the rest is roadmap.** Localized copy and any localized example chips must lead with **length** (fully supported) and keep the shot-count chip's experimental caveat ("approx.", "may be inaccurate ? filter by length instead"). Don't translate example chips into shot-count/shot-type/player queries.
3. **No silent drops ? observability is first-class.** Unsupported/empty queries are logged

loudly (`query.unavailable_request`, `query.empty_result` warn) with a correlation id; a looks-empty grid must never be silent. Any UX/i18n change to the query path must preserve this transparency.
4. **Don't over-promise NL.** Avoid calling the deterministic parser "conversational" (owner-leaning per open Q#5); the **parsed-pill** that shows the literal interpretation is the honesty mechanism ? keep it, and make it translatable.
5. **The console is job-tethered + preview-limited.** UX must not imply a multi-video library/home, per-rally video preview, login, or a true timeline scrubber ? none exist (D9/D10; P8/P9 gated). Only the **final compiled reel** previews.
6. **Privacy / consent / contact-only (Request-Access form).** Store only what's submitted + coarse country (`request.cf.country`), **no raw IP**; an explicit **consent checkbox** for alpha emails; data is **operations/contact only, never a training source** (same guardrail as the engine repo). Localized form copy must carry the consent line and the privacy posture faithfully.
7. **No-minors / consent posture (inherited engine guardrail).** The engine-side committed guardrails exclude minors from any training pool and require separate explicit opt-in for per-user training data; the form data is contact-only and never training. *(Note: the no-minors guardrail is stated in the engine repo's CLAUDE.md, not restated in these six khelsutra docs ? flagged here because the prompt asked, but it is not independently documented within the files I was given.)*
8. **"Default works / BYO-training optional" message.** *(Caveat: this exact phrasing does not appear verbatim in the six docs read.)* The faithful equivalent supported by these docs: the product runs out-of-the-box on the default (Gemini) detector path with **no setup required from the user**, while **BYO-compute (your own GPU) and BYO-storage (Drive/OneDrive/bucket/local) are optional** forks surfaced by the form's GPU/storage questions and the appliance architecture. UX/i18n should frame BYO as optional, not required, and not promise per-user model training (that's gated and consent-bound on the engine side).

Files (all absolute):
- `C:/Users/avidu/Projects/khelsutra/docs/PER_RALLY_SELECTION.md`
- `C:/Users/avidu/Projects/khelsutra/docs/REQUEST_ACCESS_FORM.md`
- `C:/Users/avidu/Projects/khelsutra/docs/README.md`
- `C:/Users/avidu/Projects/khelsutra/README.md`
- `C:/Users/avidu/Projects/khelsutra/NEXT_STEPS.md`
- `C:/Users/avidu/Projects/khelsutra/site/README.md`

Honesty caveats on this brief: (a) `git pull` could not fast-forward ? local copy may lag remote; (b) the "default works / BYO-training optional" phrasing and the explicit "no-minors" posture are **not verbatim** in these six files ? I gave the faithful, doc-supported equivalents and flagged where the source is the engine repo's guardrails rather than these docs.